# Lab Assignment 4: VERILOG OPERATORS

**Module: Verilog & Logic Synthesis**

**Overview:**

*This assignment challenges your mastery of Verilog operator semantics, bridging the gap between theoretical syntax and practical hardware inference. You will evaluate complex expressions and design a specialized combinational ALU to solidify your foundation for advanced digital design.*

## Part 1: Advanced Theoretical Assignment

### Question 1: The Concatenation and Sign Extension Trap

In Verilog, concatenations always evaluate as unsigned, regardless of the signedness of their internal operands. Evaluate the following RTL snippet designed for a data padding module:

```
module test_shift;
    wire signed [3:0] a = 4'sb1010;
    wire [7:0] result;

    assign result = {2{a}} >>> 2;

    initial begin
        $display ("Result: %b", result); // Display the result in binary format
    end
endmodule
```

- Determine the exact 8-bit binary value of result.
- Explain why the arithmetic right shift (>>>) operator fails to perform proper sign extension.
- Rewrite the assign statement so that proper 8-bit sign extension occurs.

### Question 2: Logical vs. Bitwise in Low-Power Masking

A common technique in low-power design is to`mask_enable_logical` mask data buses to prevent unnecessary toggling. A student writes the following control logic to determine if a data bus should be masked:

```
module tb;
reg [3:0] active_lanes = 4'b0100;
reg [3:0] fault_masks  = 4'b0010;
wire [3:0]  = (active_lanes && fault_masks);
wire [3:0] mask_enable_bitwise = (active_lanes & fault_masks);

initial begin
    $display("Mask Enable Logical: %b", mask_enable_logical);
    $display("Mask Enable Bitwise: %b", mask_enable_bitwise);
end

endmodule
```

A) What is the result of mask_enable_logical? 1

B) What is the result of mask_enable_bitwise? 0000

C) Explain why using logical operators (&&, ||) on multi-bit vectors for hardware gating conditions is a dangerous design practice compared to bitwise reductions. first the && or || operators evaluate as boolean converting both vectors before comparison to a 1 bit 1 or 0 logic

second in synthesis you are forcing the tool to create a massive number of gates for each vector to convert them to the one bit 1 or 0 before comparing wich creates a huge delay messing with STA

**Question 3: Operator Precedence and X-Propagation**

Evaluate the following compound expression involving equality and reduction operators. X represents an unknown state.

```
module tb;
wire [3:0] data = 4'b10X1;
wire [3:0] ref  = 4'b1001;
wire [3:0] A = 4'b1010;
wire [3:0] B = 4'b0101;
wire [3:0] C = 4'b1100;

wire       flag = ^data == (data === ref) ? 1'b1 : 1'b0;
wire [3:0] Y = A + B << 1 > C ? A ^ B : A | B;
wire [7:0] Result = { {3{^C}}, |B, ~&A, A[2:1] };

initial begin
    $display("flag = %b, Y = %b, Result = %b", flag, Y, Result);
end

endmodule
```

y = 4'1111 << 1> c ? 1111 : 1111
y = 4'b1110 > 4'b1100 ? 1111:1111
y = 1 ? 1111 : 1111
y= 1111

0

result = {000 , 1, 1, 01}
result = 0001101
result = 00001101

A) Break down the evaluation step-by-step strictly according to Verilog operator precedence.
B) What is the final value of flag, Result and Y?
C) Why is the identity operator (===) forbidden in synthesizable RTL meant for an FPGA, and how would standard hardware synthesize data == ref?

flag = x == 0 ? 1'b1 : 1'b0
flag = x ? 1'b1 : 1'b0
flag = x

=== is forbidden because it is non synthesizable
data == ref will compare both variables using xnor gates on each bit then pushing them to an and gate

**Question 4: Conditional operator and the if-else statement.**

Consider the following Verilog module and its testbench snippet:

```
module conditional_trick (
    input wire sel,
    input wire a,
    input wire b,
    output wire out_ternary,
    output reg out_if
);

    // Implementation 1: Conditional (Ternary) Operator
    assign out_ternary = sel ? a : b;

    // Implementation 2: If-Else Statement
    always @(*) begin
        if (sel)
            out_if = a;
        else
            out_if = b;
    end

endmodule
```

**Scenario:** During simulation, the inputs are driven to the following values:

- a = 1'b1
- b = 1'b0
- sel = 1'bx (unknown)

out_ternary = 1'bx | out_if = 1'b0

A) What are the simulated values of out_ternary and out_if?
B) If we change b to 1'b1 (so both a and b are 1'b1 while sel remains 1'bx), what are the new simulated values of out_ternary and out_if?

out_ternary = 1'bx | out_if = 1'b1

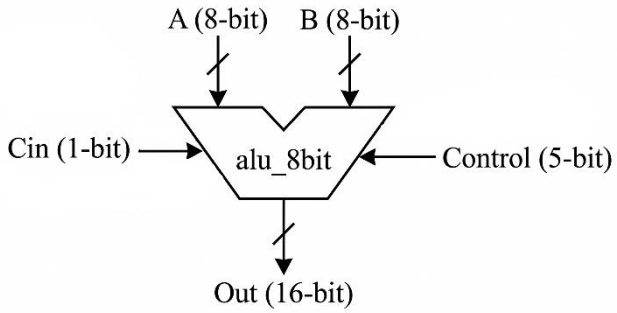
## Part 2: Comprehensive Lab (1): 8-Bit Multi-Function ALU

This lab requires you to design, implement, and verify a completely combinational 8-bit Arithmetic and Logic Unit (ALU).

You must construct a module named **alu_8bit**.

1. **Design Specifications:**
   - **Ports:**
     a. **Inputs**
        - An 8-bit **A**.
        - An 8-bit **B**.
        - A 1-bit **Cin**.
        - A 5-bit **Control**.

     b. **Outputs**
        - A 16-bit **Out**.




2. **Operational Requirements:**

   You must use a **case statement** within an **always block** that checks the Control input and acts on A, B, and Cin according to the following strict routing rules:

| ALU Functionality | | | Control |
|---|---|---|---|
| **Operation** | **description** | **Notes** | |
| ADD | A + B | | "00000" |
| ADD with Carry | A + B + 1 | | "00001" |
| SUB | A + not B + 1 | | "00010" |
| SUB with borrow | A + not B | | "00011" |
| Increment | INC A | | "00100" |
| Decrement | DEC A | | "00101" |
| Multiply and accumulate (MAC) | A*B+1 | | "00110" |
| AND | A AND B | | "00111" |
| OR | A OR B | | "01000" |
| XOR | A XOR B | | "01001" |
| NOT | NOT A | | "01010" |
| A NAND B | A NAND B | | "01011" |
| Shift left logical | A SLL B | Use the concatenation operator | "01100" |
| Shift right logical | A SRL B | Use the shift operator | "01101" |
| Shift left arithmetic | A SLA B | Use the shift operator | "01110" |
| Shift right arithmetic | A SRA B | Use the concatenation operator | "01111" |
| Rotate right | A ROR B | Use the concatenation operator. Display at the least significant byte of OUT and pad the rest by 0 | "10000" |
| Rotate left | A ROL B | Use the concatenation operator. Display at the least significant byte of OUT and pad the rest by 0 | "10001" |
| Transfer | Greater (A, B) | Display the greater of A and B (if equal, display any) at the least significant byte. | "10010" |
| Select between A, B | MUX (A, B) | Out= A when Cin=0, Out= B when Cin=1. | "10011" |
| Complement | Not A, Not B | Display the complement of A at the 1st byte and the complement of B at the 2nd byte. | "10100" |
| Compare | Compare A and B | Display the least 6 bits for greater than or equal, greater than, equal, less than or equal, less than, not equal | "10101" |
| Parity | Odd (A), Even (B) | Display the Odd parity (A) at the 1st bit and the Even parity (B) at the 2nd bit. | "10110" |

**Verification Task**

Write a **comprehensive testbench** that instantiates the ALU and drives the inputs to test all control operations. You must include edge cases in your testbench. Observe and document the outputs on the waveform window and transcript.

## Part 3: Comprehensive Lab (2) – Data Scrambler

**Objective:**

Design a fully combinational Data Scrambler and Router used in a System-on-Chip (SoC) environment to package data before crossing a clock domain. This forces the use of concatenation, reduction, conditional, and bitwise operators in a realistic scenario.

### 1. Design Specifications

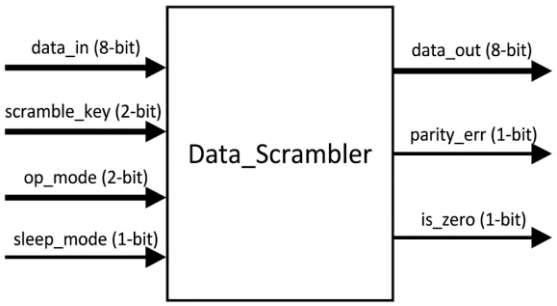
Design a module named **Data_Scrambler** with the following ports:

**Inputs:**

- **data_in**: 8-bit vector (Treat as a signed 2's complement value for arithmetic operations).
- **scramble_key**: 2-bit vector.
- **op_mode**: 2-bit control vector.
- **sleep_mode**: 1-bit active-high power control.

**Outputs:**

- **data_out**: 8-bit vector.
- **parity_err**: 1-bit output.
- **is_zero**: 1-bit output.




### 2. Operational Requirements

You must implement the logic completely combinationally using assign statements and conditional operators (?:), if statement, case statement, or an always @(*) block.

- If **sleep_mode is 1**, **data_out must be forced to exactly 8'b00000000** to prevent downstream toggle propagation (low-power gating), and **parity_err must be 0.**
- **Operating Modes** (When **sleep_mode is** 0):
  Use op_mode to determine how data_out is generated:

| op_mode | Operating Modes | data_out | Note |
|---|---|---|---|
| **2'b00** | **(Pass-through):** | data_out equals data_in | |
| **2'b 01** | **(Scramble):** | data_out is the bitwise XOR of data_in and a replicated version of scramble_key | use the replication operator {n{}} to expand the 2-bit key to 8 bits before the XOR operation. |
| **2'b 10** | **(Arithmetic Scale):** | data_out is data_in arithmetically shifted right by 2 bits (>>>). | You must ensure the sign bit of data_in is preserved during the shift or use {{}} |
| **2'b 11** | **(Nibble Swap):** | data_out swaps the upper 4 bits and lower 4 bits of data_in. | You must strictly use the concatenation operator {} for this |

- **Global Flags (Always Active when awake):**
  - **parity_err:** Must be 1 if the final data_out has an even number of 1s (Even Parity). You must use unary reduction operators.
  - **is_zero:** Must be 1 if every bit in data_out is 0. You must use the logical NOT (!) or a reduction NOR operation.

**Deliverables for Lab:**

1. **RTL Source File:** The clean, synthesizable **Data_Scrambler.v** module.
2. **Testbench Source File:** A comprehensive testbench (**tb_ Data_Scrambler.v**) that tests all four operational modes, validates sign-extension on negative numbers during Mode 10, and verifies the sleep_mode override.
3. **Waveform Screenshot:** A screenshot from your simulator showing a successful run of your testbench edge cases.

**NOTE:**
**HOW TO USE CASE STATEMENT IN VERILOG:**

- https://chipverify.com/verilog/verilog-case-statement

**Deliverables**

*Submit your **report** containing **all the answers** and your **.v files** containing all requested modules, alongside a **brief testbench (tb.v)** simulating the modules to verify correct operation.*